Protocols as Types: Composable Session Types in Prologos

The Prologos Project - from Zachary Larson

2026-03-05

Contents

1 The Insight **2**

2 Composition Patterns **2**
2.1 Pattern 1: Phase Transition (Sequential Composition) 3
2.2 Pattern 2: Mode Branching 3
2.3 Pattern 3: Inline Prefix with Named Continuation 4

3 What This Means: Protocol Libraries **4**

4 The State Machine Interpretation **5**

5 The `end` Question: Intentional Design, Not Missing Feature **6**
5.1 Why End-Splicing Is Rejected 6
5.2 The Intentionality Principle 7
5.3 Making Protocols Composable 7

6 Capability Composition **8**

7 Relationship to Existing Type System Features **8**
7.1 Session Types as Layer 5 9
7.2 Decomplection of Protocol and Process 9
7.3 Duality Composes 9
7.4 Progressive Disclosure 9

8 Theoretical Foundations **9**
8.1 Curry-Howard for Linear Logic (Toninho, Caires, Pfenning, 2011) . . . 10
8.2 Polymorphic Session Types (Caires et al., 2013) . . . 10
8.3 Context-Free Session Types (Thiemann & Vasconcelos, 2016; Padovani, 2017) . . . 10
8.4 Scribble (Honda, Yoshida, et al.) . . . 10
8.5 What Is Novel . . . 10

9 Design Principles for Protocol Composition **11**

10 Relationship to Other Principles **12**

11 Implications for the Language **12**
11.1 Protocol Libraries Become Possible . . . 12
11.2 Testing Protocols in Isolation . . . 12
11.3 Error Handling at Phase Boundaries . . . 12
11.4 AI Agent Protocols . . . 13

12 Inspirations **13**

1 The Insight

Session types are types. Continuations are type positions. Named session types can appear in continuation positions. Therefore: **protocols compose through the type system with no additional mechanism.**

A session type describes a communication protocol. After each operation (send, receive, select, offer), there is a continuation — a type describing "what comes next." When that continuation is a named session type, the protocol **transitions into a different protocol**. The type checker verifies the transition. The channel remains the same; only the protocol changes.

This is not syntactic sugar, not a macro expansion, not a runtime dispatch. It is the natural consequence of session types being first-class types in a dependently-typed language. Composition falls out of the type system for free.

2 Composition Patterns

Three composition patterns emerge from this insight, none requiring new mechanisms beyond what session types already provide.

2.1 Pattern 1: Phase Transition (Sequential Composition)

A session type's continuation names the next phase of the protocol:

```
session Handshake
  ! Version . ? Version . Authenticated  ;; after handshake → authenticate

session Authenticated
  ! Credentials
  ? [Result Token AuthError]
  +>
    | :ok   -> Ready             ;; success → operational phase
    | :fail -> end               ;; failure → done

session Ready
  +>
    | :query -> ! String . ? [Result Rows DbError] . Ready
    | :close -> end
```

The protocol flows: `Handshake → Authenticated → Ready → (loop) → end`. Each phase is a named type. Each transition is verified by the type checker. The **author** of each session type decides what comes next — `Handshake` declares "after me comes `Authenticated`," not the caller.

2.2 Pattern 2: Mode Branching

Choice operators (`+>` / `&>`) can branch into different sub-protocols:

```
session DatabaseSession
  +>
    | :query  -> QueryMode
    | :admin  -> AdminMode
    | :close  -> end

session QueryMode
  ! String . ! [List Param] .
  ? [Result [List Row] DbError] .
  DatabaseSession                    ;; return to main protocol

session AdminMode
  +>
```

```
  | :vacuum -> ? [Result Unit DbError] . DatabaseSession
  | :backup -> ! Path . ? [Result Unit DbError] . DatabaseSession
```

This is **mutual recursion between protocols**. `DatabaseSession` branches to `QueryMode`, which does its work, then transitions back to `DatabaseSession`. The protocol graph can be arbitrarily complex — any directed graph of named session types is expressible.

2.3 Pattern 3: Inline Prefix with Named Continuation

Operations are written inline, with a named type taking over as the final continuation:

```
session SecureFileSession
  ! Version . ? Version .                        ;; inline handshake operations
  ! Credentials . ? [Result Token AuthError] . ;; inline auth operations
  FileOps                                        ;; named type takes over
```

This is the most direct form of "do these things, then become that protocol." The inline prefix is protocol-specific; the named continuation is reusable.

3 What This Means: Protocol Libraries

Composition through types means session types become **building blocks**. A protocol library is a collection of named session types, each describing a reusable phase of interaction:

```
;; --- Protocol building blocks ---

session Handshake
  ! Version . ? Version . Authenticated

session Authenticated
  ! Credentials . ? [Result Token AuthError] .
  +> | :ok -> Ready
     | :fail -> end

;; --- Domain protocols built from blocks ---
```

```
session Ready
  +>
    | :query -> QueryMode
    | :close -> end

session SecureDbSession
  Handshake               ;; starts with handshake → authenticated → ready
  ;; (the chain follows from Handshake's continuation declarations)
```

Each block is independently testable, independently verifiable, and independently documentable. Complex protocols are assembled from simple, well-understood components.

4 The State Machine Interpretation

Named session types are states. Operations within a session type are transitions. The continuation after each operation names the next state.

```
                Handshake

          ! Version . ? Version


             Authenticated

        ! Credentials . ? Result



           Ready      end



QueryMode      end


 (returns to Ready)
```

This is a typed, verified finite state machine. Every transition is type-checked. Dead states are impossible (the type checker rejects protocols that

don't reach `end`). Invalid transitions are type errors. The session type **is** the state machine specification.

5 The `end` Question: Intentional Design, Not Missing Feature

A natural question arises: if `Handshake` ends with `end`, can we compose it with `Auth` by replacing `end` with `Auth`?

```
;; Both are complete, terminal protocols:
session Handshake
  ! Version . ? Version . end

session Auth
  ! Credentials . ? Token . end

;; Can we write this?
;; session Composed = Handshake . Auth    ;; hypothetical end-splicing
```

This would require **end-splicing** — a type-level operation that substitutes `end` in one protocol with the start of another. Prologos **deliberately does not do this**, for principled reasons.

5.1 Why End-Splicing Is Rejected

End-splicing introduces semantic complications:

1. **Multiple exit points**: A protocol with several branches reaching `end` would have all of them spliced. Is that always what the author intended?

2. **Recursive protocol interaction**: If a protocol has recursive self-references (`rec`) and `end` branches, splicing `end` must create a new recursive type where the self-references point to the spliced version. This requires type-level rewriting of recursive bindings.

3. **Mixed continuations**: If some branches end with `end` and others with named types, which `end`s get spliced? The semantics become ambiguous.

4. **Retroactive composition**: A protocol defined with `end` was designed as terminal. Retroactively composing it changes its meaning without changing its definition. This violates referential transparency at the type level.

5.2 The Intentionality Principle

When `Handshake` says `end`, it means "*I am done — the connection is finished.*" When `Handshake` says `. Authenticated`, it means "*I am a phase, and after me comes authentication.*" These are different protocols with different meanings.

A handshake-then-close and a handshake-then-authenticate are not the same handshake with different suffixes. They are different commitments. The named-continuation approach makes protocol composition **intentional**:

- The **author** of a session type declares what comes next
- A protocol defined with `end` is **complete** — it is not a composable prefix
- A protocol defined with a named continuation is **open** — it transitions to another protocol
- This distinction is explicit, visible in the type definition, and enforced by the type checker

This is analogous to the difference between a function that returns `A` and a function that takes a continuation `(A -> B) -> B`. Both are useful; neither is the other.

5.3 Making Protocols Composable

If a protocol author intends their protocol to be used as a composable phase, they define it with a named continuation:

```
;; Reusable: author explicitly says "something comes after me"
session Handshake
  ! Version . ? Version . Authenticated     ;; open: transitions to Authenticated

;; Terminal: author explicitly says "I'm the end"
session HandshakeDone
  ! Version . ? Version . end               ;; closed: terminates
```

Two types, two intentions, both clear. No machinery needed to distinguish them. The author's intent is visible in the type definition.

6 Capability Composition

When protocols compose, capabilities compose with them. Each named session type can carry capability requirements. The composed protocol's requirements are the union of its constituents':

```
;; Each phase has its own capability requirements
session Handshake {net :0 TlsCap}
  ! ClientHello . ? ServerHello . Authenticated

session Authenticated {net :0 AuthCap}
  ! Credentials . ? Token . Ready

session Ready {fs :0 ReadCap}
  +>
    | :read -> ! Path . ? String . Ready
    | :done -> end
```

A process implementing the full `Handshake → Authenticated → Ready` chain requires `{net :0 TlsCap, net :0 AuthCap, fs :0 ReadCap}` — the union of all phase requirements. The compiler computes this automatically. At runtime, `:0` capabilities are erased — zero cost for authority verification.

This means:

- **Protocols compose** — named types in continuation position
- **Capabilities compose** — union of per-phase requirements
- **Types verify the whole thing** — the compiler checks authority at every phase transition
- **Runtime cost is zero** — `:0` authority proofs are erased

7 Relationship to Existing Type System Features

Protocol composition through types is not an isolated feature. It connects to and reinforces several existing Prologos design principles:

7.1 Session Types as Layer 5

The Layered Architecture (DESIGN$_{\text{PRINCIPLES}}$ §Layered Architecture) places protocols at Layer 5 (`session`) and processes at Layer 6 (`defproc`). Protocol composition works entirely within Layer 5 — it is a property of protocol specifications, independent of process implementation. A `defproc` implements one endpoint of a (possibly composed) protocol; it does not need to know whether the protocol was assembled from multiple named session types.

7.2 Decomplection of Protocol and Process

DESIGN$_{\text{PRINCIPLES}}$ §Decomplection establishes that protocol specification (`session`) is decoupled from process implementation (`defproc`). Protocol composition deepens this decomplection: complex protocols can be assembled from simple building blocks at the type level, without any corresponding complexity in the process implementation. The process sees a single session type; whether it was composed from three named types or defined monolithically is invisible.

7.3 Duality Composes

Session type duality is preserved through composition. If `Handshake` transitions to `Auth`, then `dual Handshake` transitions to `dual Auth`. The type checker verifies duality for the entire composed protocol, not just each phase independently. This ensures end-to-end protocol safety.

7.4 Progressive Disclosure

Protocol composition is a Tier 3 feature — expert users building complex concurrent systems. Tier 1 users (`defn main`) never see session types. Tier 2 users (`with-open`) interact with single, monolithic protocols. Only users who need multi-phase, multi-protocol architectures encounter composition. The complexity is opt-in, following the Disappearing Features principle (ERGONOMICS §Guiding Principle).

8 Theoretical Foundations

Protocol composition through types has roots in several lines of research, but Prologos's specific combination is novel.

8.1 Curry-Howard for Linear Logic (Toninho, Caires, Pfenning, 2011)

Session types correspond to linear logic propositions under the Curry-Howard correspondence. Protocol composition corresponds to the **cut rule** — the most fundamental operation in logic. Sequencing protocols is logically equivalent to composing proofs. The theoretical foundation for what Prologos does is as old as linear logic itself.

8.2 Polymorphic Session Types (Caires et al., 2013)

Session types parameterized by other types, including continuation types. This gives the formal machinery for open-ended protocols: `Handshake[K] = ! Version . ? Version . K`. Prologos achieves the same effect through named continuations without requiring explicit type parameters.

8.3 Context-Free Session Types (Thiemann & Vasconcelos, 2016; Padovani, 2017)

Extensions that allow session types to express context-free languages, not just regular languages. This gives more expressive composition at the cost of decidability for some properties. Prologos stays within the regular fragment (tail-recursive protocols) for decidability guarantees.

8.4 Scribble (Honda, Yoshida, et al.)

A protocol description language for multiparty session types that supports protocol modules. The closest existing practical tool to protocol composition. But Scribble is a specification language — protocols are described, not typed. Prologos makes protocols first-class types in the programming language itself.

8.5 What Is Novel

The **combination** is novel:

1. Named session types as composable building blocks within a programming language (not a separate specification language)
2. Capability composition tracking alongside protocol composition
3. Propagator-based runtime that schedules composed protocols efficiently
4. Progressive disclosure — composition is opt-in, not forced

5. The intentionality principle — `end` means terminal, named continuations mean composable, and the distinction is a design choice

No existing language or tool unifies all five. The individual pieces are well-understood; the synthesis is new.

9 Design Principles for Protocol Composition

1. **Protocols Are Types.** A session type is a first-class type. It can appear anywhere a type can appear — as a continuation, as a type parameter, as a constraint. Protocol composition is type composition.

2. **Composition Is Intentional.** The author of a session type decides whether it is composable (named continuation) or terminal (`end`). Composition is not retroactive. The type definition is a commitment.

3. **Named Continuations, Not End-Splicing.** Protocols compose through named types in continuation position. There is no mechanism to replace `end` in a complete protocol with a continuation. The distinction between open and closed protocols is part of the protocol's meaning.

4. **Capabilities Compose With Protocols.** When protocols compose, their capability requirements compose via union. The compiler tracks authority through every phase transition. Authority verification is free (`:0` erased).

5. **Duality Is Preserved.** Protocol composition preserves the duality invariant. If server-side composes phases A $\rightarrow$ B $\rightarrow$ C, the client-side sees dual(A) $\rightarrow$ dual(B) $\rightarrow$ dual(C). End-to-end safety is guaranteed.

6. **State Machines Are Types.** The protocol graph (named session types as states, operations as transitions) is verified by the type checker. Dead states, invalid transitions, and stuck channels are type errors.

7. **Progressive Disclosure.** Simple protocols are simple to define. Composition is available when needed but never required. A monolithic `session` that does everything inline is always valid.

10 Relationship to Other Principles

Document	Relationship
`DESIGN_PRINCIPLES.org`	"Correctness Through Types" — protocol composition is type co
`LANGUAGE_VISION.org`	"Protocol correctness" (§What Problem Does Prologos Solve?) ex
`CAPABILITY_SECURITY.md`	"Session Types and Capability Protocols" (§Session Types) — ca
`ERGONOMICS.org`	"Disappearing Features" — composition is Tier 3, invisible to beg
`LANGUAGE_DESIGN.org`	"Session Types as Protocol Specifications" (§Session Types) — co
`PATTERNS_AND_CONVENTIONS.org`	Session type naming patterns extend to composed protocols. Eac
IO Library Design V2	The IO design uses protocol composition extensively: `Handshake`

11 Implications for the Language

11.1 Protocol Libraries Become Possible

Named, composable session types enable a **standard library of protocols** — reusable communication patterns that users can compose into application-specific protocols. Examples:

- `Handshake` — version negotiation
- `TlsHandshake` — TLS setup
- `Auth` — credential exchange
- `Heartbeat` — keepalive with timeout
- `Pagination` — cursor-based result traversal

These would live in `prologos.core.protocols` and compose freely.

11.2 Testing Protocols in Isolation

Each named session type can be tested independently. A test for `QueryMode` doesn't need to set up the full `DatabaseSession → QueryMode` chain — it directly tests the `QueryMode` protocol against a mock endpoint. This mirrors how pure functions are tested independently of their callers.

11.3 Error Handling at Phase Boundaries

Errors at phase transitions — authentication failure, handshake mismatch — are type-level events. A branch to `end` at the wrong phase is visible in the type. Error protocols can be composed just like success protocols:

```
session Authenticated
  ! Credentials . ? [Result Token AuthError] .
  +>
    | :ok   -> Ready
    | :fail -> ErrorRecovery           ;; transition to error protocol

session ErrorRecovery
  ! RetryRequest . ? [Result Token AuthError] .
  +>
    | :ok    -> Ready
    | :fail  -> end                     ;; final failure
```

11.4 AI Agent Protocols

Multi-agent communication (LANGUAGE$_{\text{VISION}}$ §AI Agent Infrastructure) benefits directly from protocol composition. Agent interaction protocols are built from composable phases:

```
session AgentNegotiation
  ! Proposal . ? CounterProposal .
  +>
    | :accept -> CollaborativeWork
    | :counter -> AgentNegotiation     ;; re-negotiate
    | :reject -> end

session CollaborativeWork
  rec
    +>
      | :delegate -> ! Task . ? Result . CollaborativeWork
      | :report   -> ! Summary . end
```

The session type guarantees that agents follow the negotiation protocol, that delegation respects authority boundaries (via capabilities), and that the collaboration terminates.

12 Inspirations

- **Toninho, Caires, Pfenning** — Session types as propositions (Curry-Howard for linear logic). Protocol composition = cut rule. The deepest theoretical foundation for this work.

- **Honda, Yoshida, Carbone** — Multiparty session types and the Scribble protocol description language. The closest practical precedent for protocol modularity.
- **Lindley & Morris** — Semantics for propositions as sessions. The cut rule gives composition; our named continuations are a user-facing expression of the same operation.
- **Padovani** — Context-free session types. Shows that session types can express more than regular languages, though we stay within the regular fragment for decidability.
- **Mark Miller (E language)** — Object-capability model. Capabilities compose through delegation; our capability composition through protocol composition mirrors this at the type level.